

NVR: Vector Runahead on NPUs for Sparse Memory Access

Hui Wang^{1*}, Zhengpeng Zhao^{4*}, Jing Wang¹, Yushu Du¹, Yuan Cheng^{2,3}, Bing Guo¹, He Xiao⁵,
Chenhao Ma¹, Xiaomeng Han¹, Dean You¹, Jiapeng Guan⁶, Ran Wei⁶, Dawei Yang^{2,3}, Zhe Jiang¹✉

¹National Center of Technology Innovation for EDA, School of Integrated Circuits, Southeast University ²Houmo AI

³Nanjing University ⁴Huazhong University of Science and Technology ⁵Harbin Institute of Technology ⁶Dalian University of Technology
whmio0115@seu.edu.cn, u202114911@hust.edu.cn, yuancheng@mail.nju.edu.cn, dawei.yang@houmo.ai, zhejiang.uk@gmail.com

Abstract—Deep Neural Networks are increasingly leveraging sparsity to reduce the scaling up of model parameter size. However, reducing wall-clock time through sparsity and pruning remains challenging due to irregular memory access patterns, leading to frequent cache misses. In this paper, we present NPU Vector Runahead (NVR), a prefetching mechanism tailored for NPUs to address cache miss problems in sparse DNN workloads. Rather than optimising memory patterns with high overhead and poor portability, NVR adapts runahead execution to the unique architecture of NPUs. NVR provides a general micro-architectural solution for sparse DNN workloads without requiring compiler or algorithmic support, operating as a decoupled, speculative, lightweight hardware sub-thread alongside the NPU, with minimal hardware overhead (under 5%). NVR achieves an average 90% reduction in cache misses compared to SOTA prefetching in general-purpose processors, delivering 4x average speedup on sparse workloads versus NPUs without prefetching. Moreover, we investigate the advantages of incorporating a small cache (16KB) into the NPU combined with NVR. Our evaluation shows that expanding this modest cache delivers 5x higher performance benefits than increasing the L2 cache size by the same amount.

I. INTRODUCTION

Emerging Deep Neural Networks (DNNs), particularly Large Language Models (LLMs), often scale to hundreds of billions of parameters [1], increasingly consuming more storage, memory bandwidth, and computational resources. Fortunately, these workloads are typically over-parameterised [2], where up to 90% of parameters in prevalent models can be pruned while maintaining comparable performance [3]. This redundancy presents an opportunity to leverage sparsity to reduce such intensive resource demands.

Theoretically, more fine-grained sparsity patterns yield higher acceleration by skipping more zero-valued elements. However, as shown in Fig. 1 (a), these patterns introduce unstructured characteristics when directly skipping zero values randomises vector access order, leading to irregular memory access patterns. These irregular patterns frequently result in cache misses, which not only waste main memory bandwidth but also degrade NPU performance. Taking sparse KVCache in LLM workloads [4] as an example, Fig. 1 (b) illustrates that despite a 16x parameter reduction, frequent cache misses lead to extensive main memory interactions. This results in only a 5x actual speedup, substantially below theoretical expectations. Cache misses thus emerge as a key barrier to sparse workloads.

Existing Work. Recent research has focused on mitigating the side effects of fine-grained sparsity. Farshchi et al. [5] propose a bitmask sparse data format in NVDLA, while Eyeriss [6] design a run-length encoding for non-zero elements. Albericio et al. [7] introduce additional mapping algorithms, regularising memory access patterns through label cache mechanisms. Meanwhile, Liu [8] et al. explored sparse domain compute units, developing a butterfly-based architecture for efficient sparse data structure processing. Whilst most of these approaches attempt to regularise memory access patterns to avoid cache misses, they often introduce substantial area and control logic overhead that outweighs their benefits; therefore, the practical applicability of these algorithm-specific are significantly restricted.

Contributions: Different from all previous works, we tackle the sparsity challenge from a different perspective. Instead of regularising irregular memory access patterns, our approach embraces the inherent characteristics of sparse computations. We leverage prefetching techniques directly to mitigate the impact of cache misses, rather

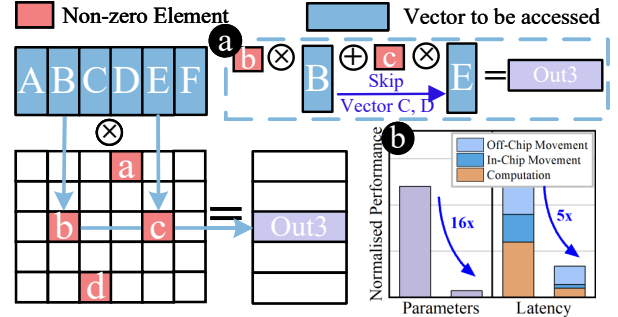


Fig. 1: Sparsity is a widely adopted approach for speedup and energy efficiency through skipping zero in processing, which introduces substantial irregular memory accesses.

than attempting to eliminate the irregularity itself, which incurs high overhead and lacks generality. We propose a vector runahead mechanism for NPU (NVR), drawing inspiration from the runahead execution model [9], [10]. NVR integrates a lightweight prefetcher decoupled with CPU and NPU, achieving significant cache miss reduction through speculative execution utilising idle NPU units. NVR is a micro-architectural solution requiring no compiler or algorithmic support, remains orthogonal to prior optimisation methods.

To this end, we implement NVR, to the best of our knowledge, the first system-level prefetching mechanism specifically designed for NPUs. Our architecture reduces L2 cache misses by 90% compared to SOTA prefetching techniques in general-purpose processors through the effective prediction of irregular access patterns, with under 5% area overhead. Furthermore, NVR reduces off-chip memory accesses by 75% during NPU execution. When combined with NSB (a small cache structure in NPUs), there is an additional 80% off-chip memory access reduction benefit. Our system-level evaluation of LLM workloads further validates the effectiveness of NVR, demonstrating an average 50% throughput improvement in IO-bound scenarios.

It's worth noting that NVR is not limited to sparse tasks. It can be applied to any operations exhibiting unstructured characteristics, resulting in irregular memory accesses.

II. BACKGROUND AND MOTIVATION

Although sparse workloads offer high theoretical memory-level parallelism (MLP), hardware systems struggle to exploit this algorithmic advantage effectively. As shown in Fig. 1 (b), we systematically profile the parameters of Double Sparsity [4] in LLMs workload and its performance on NPU with a 256KB L2 cache. We find that reducing parameters does not lead to proportional decreases in off-chip memory accesses, resulting in out-of-bounds accesses for explicit buffers (like scratchpad) or cache misses for L2 cache in the chip. As these two scenarios are essentially identical, we focus our analysis on cache misses in the following discussion.

A. Sparse Memory Accesses: Misses Are a Fact of Life

In this section, we introduce the sparse computation and analyse the sparse irregular memory access patterns, which lead to a heavy cache miss and NPU stalls. As an example, Fig. 2 (right) illustrates a typical sparse matrix multiplication (SpMM), which is common in DNNs, computation with compressed sparse row (CSR) format. The code snippet demonstrates the sparse weight matrix (w) selectively indexes

* Equal contribution.

✉ Corresponding author.

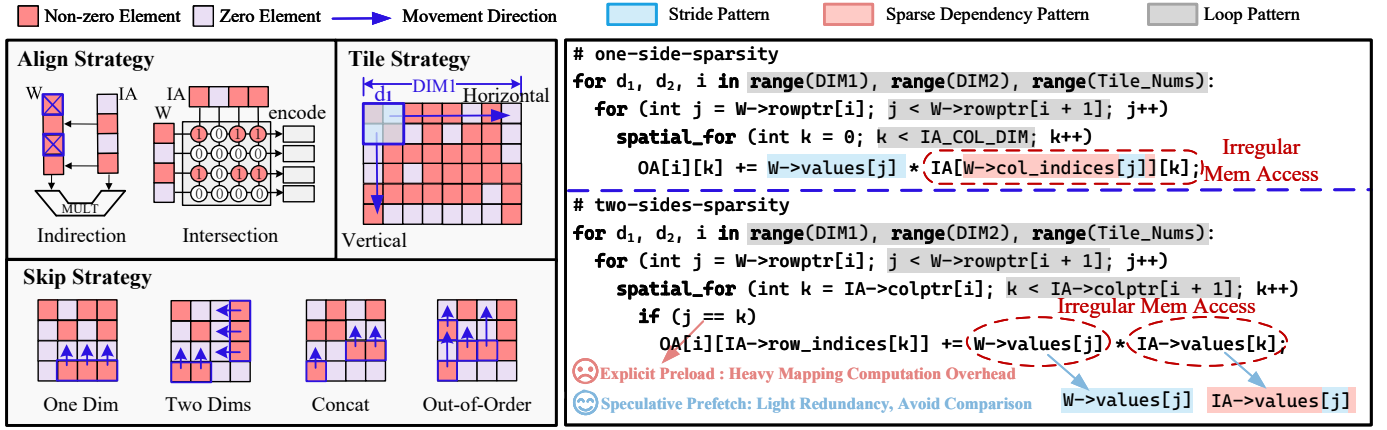


Fig. 2: Sparse Matrix Multiplication can be categorised into one-side-sparsity and two-sides-sparsity patterns, with higher sparsity offering greater speedup potential at the cost of more challenging access patterns. Here, `spatial_for` denotes parallel operation on the NPU, while `IA` (input activation), `W` (weight), and `OA` (output activation) represent the input variables, weight parameters, and output results, respectively.

input activation (`IA`) vectors for multiply-accumulate operations. This workload exhibits several critical factors contributing to cache misses: **Indirect Memory Accesses** Since only non-zero data is computed, the data must be aligned, which involves numerous index dependency chains. While `W[j]` accesses are often sequential and can be captured by streaming prefetch, `IA[sparse_func(W[i])]` accesses target non-sequential locations. Given that, `IA` typically spans a large index space that exceeds L2 cache capacity, resulting in frequent cache misses. This alignment pattern is common in across sparsity applications, like hash-table indexing and sampling operation in point cloud networks, where `sparse_func` is replaced by complex indexing schemes, often requiring dedicated processing units.

Dynamic Loop Boundaries The tiling strategy refers to reordering element arrangements for hardware computation units (e.g., systolic arrays, vector units). This manifests as irregular loop boundaries. In large dynamic routing architectures such as Mixture-of-Experts (MoE), the memory span between `rowptr[i]` and `rowptr[i+1]` can be substantial, posing significant challenges for cache capacity.

Data Shuffle As shown in Fig. 2 (left), NPUs employ various skip strategies to accelerate computation by bypassing zero elements. NPUs with stronger reordering capabilities can group farther distant data elements for unified processing. This leads to densely packed, long-stride access patterns within short time intervals, causing catastrophic cache behaviour. This issue is particularly evident in LLMs' KVCache [4], where sparse attention employs TopK selection to retain only the k highest-scoring vectors for computation. For LLMs with extended contexts, KVCache indexing can span several gigabytes, significantly exceeding cache capacity.

B. Cache-miss Vulnerable NPUs

Facing numerous cache misses, NPUs suffer even more severe performance degradation than CPUs and GPUs. While CPUs leverage Out-of-Order (OoO) execution and reorder buffers (ROB) to tolerate this through fine-grained instruction-level overlap, and GPUs utilise thread-level scheduling for cache misses hiding, NPUs remain limited in such capabilities. Built primarily on Single Instruction Multiple Data (SIMD) architectures, NPUs possess only coarse-grained instruction parallelism, with compute and memory movement channels decoupled. As demonstrated in Fig. 5 in the experiment, even with ideal OoO execution on NPUs, it also shows suboptimal performance in handling cache misses. Critically, the data-parallel nature of NPUs means a cache miss in any vector element stalls the entire processing pipeline, leading to performance degradation on sparse workloads.

C. Challenge for NPU Prefetch

There are many conventional prefetchers designed for handling irregular memory access patterns mentioned in section II-A. For sparse

workloads with abundant irregular memory accesses, simple pattern-based [11]–[13] or history-based [14]–[16] prefetchers often fail to effectively capture and predict these accesses, whereas runahead-based [9], [17], [18] prefetching has been widely adopted as the solution. Runahead employs speculative execution to hide cache miss latency by prefetching future memory accesses. VR [19] and DVR [10] explore leveraging vector units to perform runahead in parallel, which is a well-suited approach for NPUs that inherently support vector operation instructions. However, applying these techniques directly to NPUs presents several challenges.

Despite the success of these prefetching techniques in general-purpose processors, the unique characteristics of NPU architectures and their workloads necessitate a fundamentally different approach. Diverse sparse workloads on NPUs often require customised hardware implementations, such as hash-table-based methods in point cloud processing. Meanwhile, sparse workloads commonly leverage specialised sparse data formats, like TACO's [20] multi-dimensional encoding and SMASH's [21] hierarchical bitmap-based decompression, handled by dedicated processing units. This diversity of dependency chains, coupled with the need for parallel execution, poses significant challenges for prefetchers in both pattern capture and overhead. A generalised prefetching approach decoupled from specific sparse computation patterns is necessary. Moreover, NPUs operate with coarse-grained instructions that process entire vectors or matrices. The decoding of such coarse-grained instructions implies weaker locality awareness and significant computational overhead, making traditional instruction-level optimisations less applicable.

III. NVR: DESIGN PHILOSOPHY

With these challenges in mind, we propose an adaptation of runahead tailored for NPU architectures, addressing three key aspects: **Decoupled and Non-Invasive Philosophy**: Throughput is the primary metric for NPUs. To maximise it, prefetching mechanisms should avoid introducing additional control logic that stalls execution, ensuring maximum data parallelism [22]. Our design decouples from NPU computation logic, allowing simultaneous speculative execution to proactively generate memory requests. The system maintains non-invasive integration by passive monitoring the state and extracting load instruction information through read-only operations. This approach allows early initiation of memory requests without interfering with NPU execution or custom instruction requirements.

Coverage-Oriented Philosophy: In NPU vector operations, computation can proceed only when all data in the batch are ready. Our experiments shown in Fig. 8 (a) reveal that the overall cache miss rate decreases significantly faster than per-batch cache miss rates. While optimising per-batch cache misses is more challenging, it's crucial for substantial performance improvement. Consequently, our

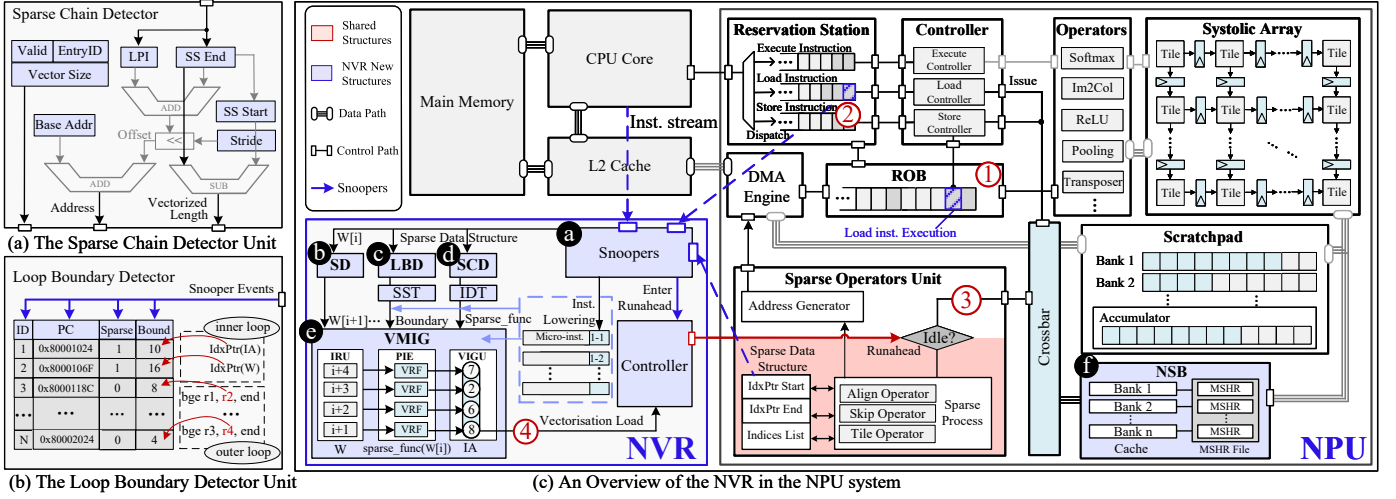


Fig. 3: NVR micro-architecture and components. Purple blocks represent NVR additions to the system. Red blocks indicate shared components between NVR and NPU, assisting speculative execution during NPU sparse unit idle periods. **b**: SD: Stride Detector; **c**: LBD: Loop Bound Detector; **d**: SCD: Sparse Chain Detector; **e**: VMIG: Vectorisation Micro-Instruction Generator; **f**: NSB: Non-blocking Speculative Buffer.

design prioritises complete batch retrieval, accepting some prefetch redundancy as a reasonable trade-off. On the other hand, combining this with fuzzy range loading has the added benefit of reducing the overhead of control logic, like branch prediction.

Micro-Instruction-Level Vectorisation Philosophy: NPUs' SIMD architecture operates with vectorised instructions, which typically could be decomposed into multiple micro-instructions spanning several cycles. This fine-grained approach enables precise handling of cache misses while detecting stronger memory access patterns in this granularity. Leveraging NPUs' native support for vector load instructions, we can bundle memory addresses without additional execution units, improving both prediction accuracy and MLP.

In summary, vector runahead on NPU has several unique features. We can summarise our runahead design into three Q&A and introduce detailed micro-architecture in the next session:

Q&A1. When to enter runahead mode: Enter runahead when a load instruction in the NPU's ROB executes, prefetching for the next load instruction in the reservation station. NVR executes in parallel with the NPU instruction stream, extracting information through snooping to generate vectorised memory requests ahead of NPU execution. By computing approximate boundaries in advance, it enables early issuance of vector load instructions.

Q&A2. Where to execute NVR: The NVR is integrated between the CPU and NPU, prefetching speculatively. To achieve decoupled and non-invasive operation, the prefetching logic is architecturally separated from the NPU pipeline.

Q&A3. How to prefetch: NVR leverages idle computational resources in NPU's sparse processing units to perform approximate dependency chain calculations in parallel. Subsequently, NVR generates native NPU vectorised load instructions for the NPU pipeline, prefetching data into the L1/L2 cache hierarchy. This approach efficiently exploits NPU's vector instructions and architectural characteristics for effective prefetching and analysis.

IV. NVR MICRO-ARCHITECTURE

We use Gemmini [23] as our baseline to demonstrate NVR's applicability to general NPUs, as it embodies typical NPU architecture. Its coarse-grained design inherently struggles with cache miss handling, making it ideal for demonstrating NVR's design and effectiveness.

A. Overall Architecture

Fig. 3 illustrates NVR's micro-architectural modifications to the baseline Gemmini configuration, which comprises an in-order core

and DNN accelerator sharing a unified L2 cache. The original Gemmini does not support sparse computation, so we have incorporated a basic implementation of the sparse function to demonstrate our prefetcher's design. As illustrated in Fig. 3 **b**, the sparse unit is primarily designed to handle alignment, skipping, and tiling of sparse data, three processing techniques discussed in Section II-A.

To support NVR, we augment the baseline with the following structures (purple blocks): A snoop unit monitoring the status of the CPU and the NPU (Fig. 3 **a**). A stride detector (SD) that tracks stream memory access patterns (Fig. 3 **b**). A loop boundary detector (LBD) that performs predictive analysis of iteration bounds for both unrolled and nested loop structures (Fig. 3 **c**). A sparse chain detector (SCD) identifies indirect memory access dependency chains and computes their corresponding memory addresses (Fig. 3 **d**). A vectorisation micro-instruction generator (VMIG) that bundles prefetch sequences with related access patterns into vectorised prefetch operations, optimising bandwidth utilisation (Fig. 3 **e**). An optional non-blocking speculative buffer (NSB), which serves as a small in-NPU cache equipped with miss status holding registers (MSHRs) (Fig. 3 **f**).

The steps of the NVR startup and execution process are numbered with red circles in Fig. 3. The NVR initiates runahead execution upon detecting new load instructions in execution within the NPU's ROB. The NVR's controller monitors CPU and NPU states via snoopers and triggers runahead request for speculative computations during NPU sparse unit idle periods. During execution, the stride detector captures patterns at index i to predict subsequent $W[i]$ values. The SCD supports this by generating indirect access address generation for $IA[sparse_func(W[i])]$ predictions. To optimise prefetch operations, the LBD analyses loop structures to prevent overruns and enables request packing. The VMIG reconstructs decomposed micro-instructions by synthesising information from these components and generates new vectorised prefetching instructions inserted into the NPU pipeline for prefetching operations. As a complementary approach, the NSB offers an optional mechanism to enhance the prefetching strategy, demonstrating a reduction in cache misses experimentally. The following sections detail these components' design.

B. Stride Detector (SD)

The stride detector is the fundamental pattern recognition unit in NVR design and primarily used to predict the next batch of addresses for $W[i]$. It continuously predicts the data memory access pattern for step patterns by stream prefetching. For $W[i]$, neural networks tend to have relatively few branch predictions in their workloads, and

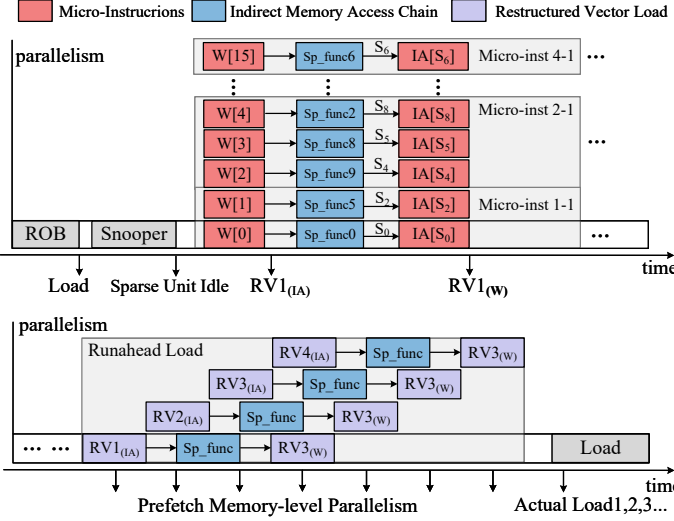


Fig. 4: Vectorisation micro-instruction generation pipeline. Micro-instruction 1-1 represents the first micro-instruction of instruction 1. Each micro-instruction loads an indeterminate number of data.

a high prediction accuracy can be achieved by recording the address differences and the confidence levels.

C. Snoopers and Controller

The snoopers are non-invasive probes used to precisely extract the architectural states of both the CPU and NPU. We monitor three critical signal types: (1) branch instructions from the CPU, which provide LBD with nested loop context information, (2) custom load-related instructions in the NPU, which are used to determine the optimal timing for runahead mode activation, and (3) NPU sparse unit registers, which supply the metadata essential for the NVR prediction mechanisms. The read-only, non-invasive design of the snoopers maintains architectural integrity by preventing any modifications to NPU computational logic and avoiding interference with the interactions between the CPU and NPU. Upon entering runahead mode, the controller sends speculative execution requests to the sparse unit. When runahead mode is triggered, the controller sends speculative execution requests to the sparse unit, enabling it to proceed with predictive computations. The system then monitors the unit availability, using the snoopers infrastructure to retrieve the necessary sparse unit data as soon as it becomes available.

D. Sparse Chain Detector (SCD)

The SCD identifies and predicts patterns in sparse computations through two critical components: historical information preservation and indirect prefetching pattern learning. An Indirect Pattern Table (IPT) is maintained to record indirect prefetch patterns, including details such as the Last Prefetch Indirect (LPI) and the sparse structure's start address. In DNN workloads, computation index patterns typically exhibit locality characteristics. Due to the large volume of data processing, indirect index patterns remain relatively stable over time intervals, often appearing as shallow indirect chains, as formulated below:

$$IA_{\text{address}} = IA_{ss_start} + (W_{LPI} \ll stride)$$

, where IA_{address} represents the predicted indirect access address, IA_{ss_start} denotes the base address at the start element of the loop iteration, and W_{LPI} indicates the value of last prefetched W . Here, $stride$ defines the offset between consecutive memory accesses. The prediction for the address of IA depends on the value of W_{LPI} .

By recording the structural information of W and IA from previous operations, SCD can effectively track sparse operation chains and predict subsequent indirect prefetching addresses via stride learning. This predictive mechanism is particularly beneficial in scenarios with repetitive sparse patterns. The mechanism requires sparse processing boundaries and current processing indices, such as CSR

$col_indices$ which locates the non-zero elements in columns and $rowptr$ for tracking row start positions from NPU operations, information that is readily available in most sparse data formats. Unlike traditional prefetchers, which lack access to NPU register-level information and often depend on statistical methods like EWMA [24], leading to inefficiencies, our approach uses the NPU's sparse unit to run ahead. This allows us to preserve and leverage historical information, significantly improving the prediction of sparse vector accesses while reducing redundant memory requests.

E. Loop Bound Detector (LBD)

DNN workloads inherently consist of multiple nested and unrolled loops, primarily involving matrix and vector operations, where higher-dimensional operations naturally require deeper loop nesting structures. To address the challenges of memory access patterns in these structures, the LBD employs systematic loop behaviour and boundary tracking to optimise prefetch sizing while preventing boundary-crossing invalid prefetches.

As illustrated in the listing of Fig. 2, nested loops typically occur during the traversal of rows and columns of a matrix, while unrolled loops are often used in parts of matrix multiplication where multiple indirect chains are executed in parallel. As depicted in Fig. 3 (b), LBD maintains historical information through a Sparse Structure Table (SST), where each parallel port has a dedicated entry to facilitate concurrent tracking of sparse chains. LBD implements a dual-mode boundary prediction, handling both sparse and normal boundaries. The mechanism distinguishes loop hierarchies from inner to outer levels through entry IDs, with each entry maintaining comprehensive loop instruction information, including PC value, boundary values, and operational mode. For standard loop boundaries (illustrated in line 1, Fig. 2 (right)), LBD captures historical boundary information by monitoring register values of jump instructions, as shown with RISC-V B-type branch instructions in Fig. 3 (b). For variable boundaries in sparse computations (shown in line 2, Fig. 2 (right)), boundary information is dynamically acquired through snoopers from sparse unit registers. Within this framework, upon detecting loop instructions, the LBD not only uses historical data to predict the boundary but also learns loop boundaries by examining input register values from comparison instructions. At the same loop level, instructions are consolidated into memory access requests, while boundary values act as crucial constraints to prevent excessive prefetching.

F. Vectorisation Micro-Instruction Generator (VMIG)

In NPU-based sparse data processing, the efficiency of the SIMD pipeline is often compromised because traditional vector instructions fail to fully utilise memory bandwidth when dealing with skipped or zero-valued elements. Strictly following NPU runtime loading instructions for prefetching would lead to significant bandwidth wastage. Instead, prefetch operations can bypass the rigid loading logic of the NPU, and the VMIG addresses this by restructuring the NPU's native load instructions at the micro-instruction level. This approach optimises sparse data patterns, leveraging existing vector load units without additional hardware support.

VMIG implements a three-stage pipeline where each stage is executed by dedicated hardware units. Initially, the Instruction Reconstruction Unit (IRU) manages the micro-instruction restructuring, using the SST provided by the LBD and integrating stride detection for speculative execution. The following stage is handled by the Parallel Inference Engine (PIE), which uses the IPT from the SCD and the Vector Register File (VRF) to predict sp_func concurrently across multiple data streams. As shown in Fig. 4, it parallelly executes sixteen consecutive dependency chains that generate W from IA . Finally, the Vector Instruction Generation Unit (VIGU) synthesises these restructured load instructions into a single vector operation, dynamically optimising the memory access.

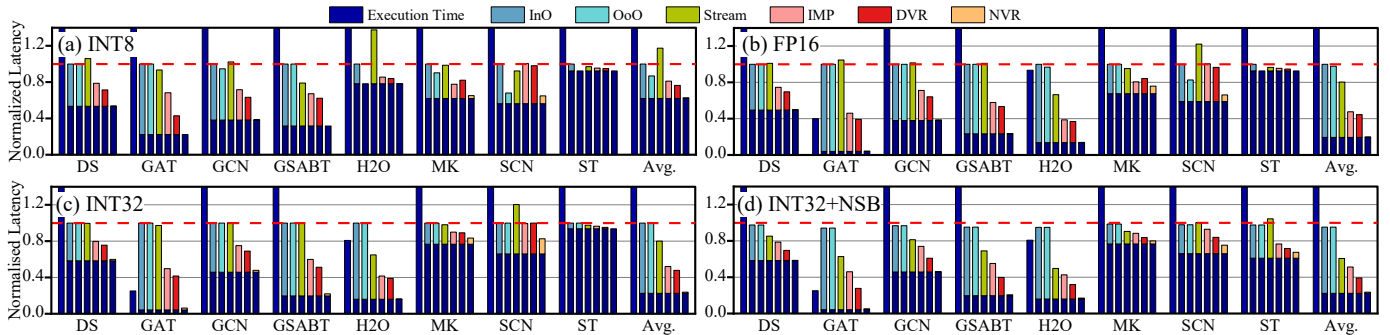


Fig. 5: Normalised wall-clock time latency for each sparse workload. Within each group, each bar from left to right denotes execution in density, in order execution, OoO execution, IMP, DVR, and NVR, respectively. The lower segment indicates the NPU base execution time, whilst the upper segment represents the stall time caused by cache misses.

This optimisation strategy leverages the NPU’s inherent SIMD load capabilities and vector registers, eliminating the need for additional vector hardware. As shown in Fig. 4, the process is fully pipelined, enhancing MLP through rapid consecutive memory request issuance. Compared to executing single instructions, this pipelined approach significantly reduces memory access latency and improves bandwidth utilisation. The efficiency also depends on the MSHR, which prevents cache miss events from blocking subsequent prefetch operations.

G. Non-blocking Speculative Buffer (NSB)

For discrete data structures, utilising scratchpad memory incurs substantial logic overhead for data transformation and retrieval operations. However, the characteristics of sparse data patterns present unique opportunities for cache-based optimisation. By strategically storing sparse discrete data in the cache while maintaining continuous data in scratchpad memory (e.g., dense vectors in one-side-sparsity), we can leverage implicit cache line reuse patterns. This cache-based approach naturally accommodates irregular reuse patterns without the costly pre-computation overhead required by scratchpad implementations. To exploit these characteristics, we introduce NSB, a compact non-blocking cache architecture optimised for discrete element management. Sparse workloads exhibit irregular memory access patterns with extensive index spaces at high sparsity, direct-mapped or low-associativity configurations frequently encounter conflict misses. Thus, we implement a high-way set-associative mapping strategy. While NSB cannot mitigate L2 cache misses during prefetch operations (as data inherently resides off-chip), it significantly reduces NPU-to-L2 latency and off-chip memory accesses during actual load instruction execution. Experimental results demonstrate that NSB further reduces bandwidth requirements by 5× compared to baseline NVR prefetching. The design incorporates an MSHR file to manage concurrent memory operations, enabling tracking of outstanding load buffer requests and cache misses. This MSHR infrastructure coalesces multiple outstanding requests to the same cache line, eliminating redundant memory accesses and optimising bandwidth utilisation.

H. Hardware Overhead

We implement NVR in RTL and synthesise our design using Synopsys Design Compiler on TSMC 28nm process technology at 2.0 GHz. The area overhead is 3% and 4.6% relative to the baseline Gemmini architecture, for configurations without and with NSB, respectively. These results demonstrate that NVR achieves its performance benefits with minimal hardware cost.

V. EVALUATION

A. Experimental Setup

We implement and evaluate NVR by integrating it into a Gemmini-like NPU model constructed using the ScaleSim simulator [25]. Additionally, we leverage LLMCompass [26], a specialised simulator designed to evaluate hardware optimisations in LLM inference.

TABLE I: Sparse Computation Workload.

Workload	Short	Domain
Double Sparsity [4]	DS	large language model
Graph Attention Networks [35]	GAT	graph neural networks
Graph Convolutional Networks [36]	GCN	graph neural networks
Graph Sparse Attention [37]	GSABT	sparse attention
Heavy-Hitter Oracle [38]	H2O	large language model
MinkowskiNet [39]	MK	point cloud
SparseConvNet [40]	SCN	point cloud
Switch Transformer [41]	ST	mixture of experts

Comparison We evaluate our approach against in-order Gemmini (serial execution of load and compute instructions), ideal out-of-order Gemmini (overlapping the load and computation time), and several prefetchers. We evaluate the following prefetch techniques: 1) stream prefetcher [27], the simplest prefetch mechanism based on stride; 2) IMP [28], one of the SOTA indirect memory prefetchers, prefetching indirect memory access at the L1 cache; 3) DVR [19], one of the SOTA runahead techniques, vectorising the same chain of indirect memory accesses across multiple invocations of an inner loop. 4) Our NPU Vector Runahead. For a fair comparison, we add the Gemmini custom ISA support for these prefetchers and expand them to the same number of parallels as the NVR.

Workload Sparsity is widely present across various workloads in DNNs. We select several key sources of sparsity, including attention mechanisms [29], [30], MoE structures [31], [32], 3D point cloud processing [33], [34], and so on. Table I presents representative workloads extracted from various models’ linear layer memory access patterns based on their functionalities and domains.

B. Performance Breakdown

Since DNNs commonly employ low bit-width computations, we evaluate different bit-width configurations (INT8, FP16, INT32) to analyse their impact on cache behaviour and system performance, as shown in Figs. 5 (a), (b), and (c). Higher data bit-width inherently requires longer data movement time and larger cache block capacities, increasing the probability of cache misses.

Overall, cache miss stalls represent a substantial portion of wall-clock time across most sparse workloads. The Switch Transformer presents a notable exception, showing lower cache miss ratios due to its relatively fixed network architecture and block-like data distribution patterns. For sparse workloads, the NPU’s OoO (out-of-order) execution capabilities struggle to effectively overlap data movement with computation to mitigate cache miss latency, as several workloads are inherently IO-bound. While prefetching mechanisms generally reduce cache miss stall time effectively, stream prefetchers occasionally introduce performance penalties due to their lower accuracy. Notably, our NVR implementation achieves reductions in latency, averaging 98.3%, 99.2%, and 97.3% for INT8, FP16, and INT32, respectively.

As shown in Fig. 5 (a), we evaluate INT32 latency variations with NSB enabled. we evaluate latency variations with NSB using INT32 configurations. The red line represents the baseline latency without NSB. With NSB enabled, L2 cache miss latency decreased by about

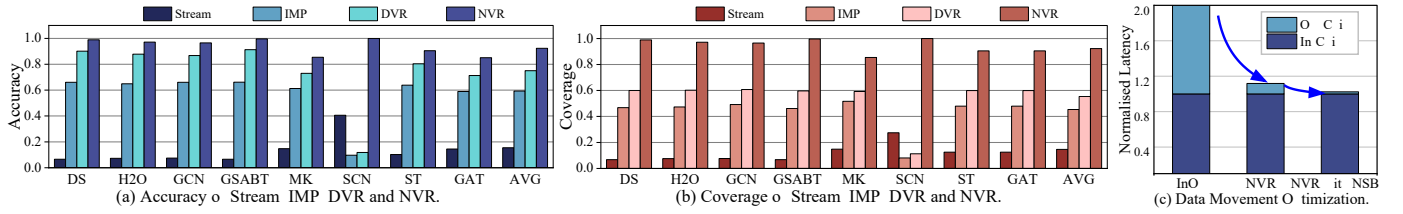


Fig. 6: Prefetcher accuracy and coverage evaluation and the data movement optimisation results of NVR.

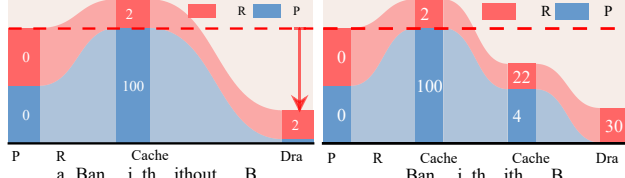


Fig. 7: Normalised bandwidth allocation without/with NSB.

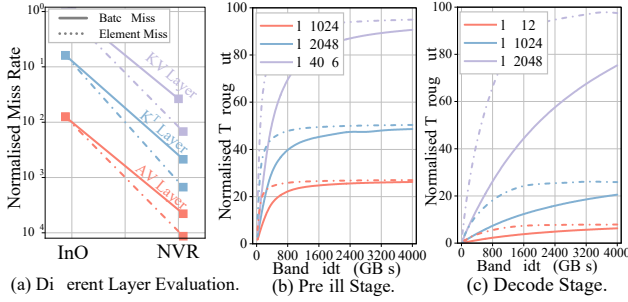


Fig. 8: System-level Evaluation on LLMs. In (b) and (c), dashed and solid lines represent NVR and baseline performance respectively, where l denotes input/output sequence length.

15%. When combined with NVR, NSB achieves a 40% reduction in cache miss stalls, but performs poorly with conventional stream prefetchers. Thus, NSB activation depends on prefetcher accuracy.

C. Prefetching Effectiveness and Off-Chip Bandwidth Reduction

This experiment evaluates prefetcher performance across sparse workloads using two key metrics: accuracy and coverage. NVR demonstrates consistently high performance, maintaining both accuracy and coverage rates above 90% across most workloads. As shown in Fig. 6 (b), coverage, which is more crucial for NPU performance, presents a greater challenge than accuracy. Benefiting from our fuzzy prefetch strategy, NVR also achieves significantly higher performance in this metric compared to other prefetchers.

The combined effect of accuracy and coverage directly influences prefetch bandwidth and the volume of off-chip memory accesses caused by cache misses during NPU execution. Figs. 7 (a) and (b) demonstrate the total bandwidth, including prefetch bandwidth. In both cases, compared to the InO baseline, the off-chip memory bandwidth is reduced by around 75%. Since the prefetch bandwidth is often more easily overlapped, Fig. 6 (c) shows the NPU actual load execution time with the memory accesses bandwidth removed. NVR can effectively reduce the off-chip memory accesses by 30x, and with NSB, it can be further reduced by 5x.

D. LLM Inference Evaluation

To validate our system-level optimisations, we evaluate the impact of NVR on the performance of LLMs. The experiments are conducted using the LLMCompass simulator. As shown in Fig. 8 (a), the NVR implementation reduces overall latency by decreasing cache miss stall time in the LLMs. Under NVR, both overall and per-batch cache miss rates decrease exponentially, with the latter showing a slower decay rate. LLMs are typically composed of two main stages: prefill and decode. It has been proven that the prefill stage is compute-bound, while the decode stage is IO-bound. Figs. 8 (b) and (c) demonstrate the benefits of applying NVR to the entire LLM network.

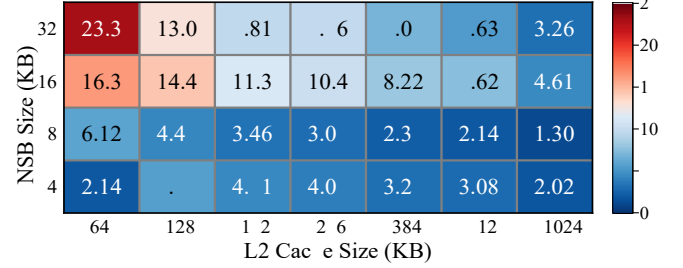


Fig. 9: NSB and L2 cache impact on performance. Performance metric: the inverse of latency and area (higher is better).

It is evident that for the compute-bound prefill stage, NVR can expedite reaching maximum throughput, particularly in low-bandwidth scenarios. For the IO-bound decode stage, the NVR architecture demonstrates an average 50% throughput improvement through reduced off-chip memory accesses. This enhancement becomes increasingly pronounced with longer output sequences.

E. Sensitivity Analyses

To evaluate NSB's efficacy, we conduct comprehensive sensitivity analyses across varying NSB and L2 cache parameters. The performance metrics represent the inverse relationship between latency and area, calculated as the product of NSB and L2 Cache dimensions, ensuring equivalent area penalties for both scaling approaches. As illustrated in Fig. 9, modest NSB buffer configurations yield substantially higher performance improvements than equivalent L2 cache scaling. Specifically, in a configuration with 256KB L2 cache and 4KB NSB, quadrupling the NSB capacity delivers 5x the performance benefit over scaling the L2 cache to 1024KB.

VI. CONCLUSION

In this paper, we present NVR, a runahead prefetching mechanism addressing the challenges of sparse DNN workloads on NPUs. This work demonstrates specialised prefetching techniques for DNN applications on custom architectures beyond general-purpose processors. **Lessons we learnt.** This work adopted a holistic, workload-driven approach to accelerate the DNNs' executions. By workload profiling and architectural analysis, key bottlenecks were identified, guiding targeted system re-architecture. Instead of simply adding more hardware resources, this work addressed fundamental inefficiencies, achieving performance gains equivalent to multiple times the benefits of hardware scaling, but with significantly reduced area and power costs. The constructed NVR provides key insights that precise, interdisciplinary optimisations rooted in a deep understanding of workload-architecture interactions. We believe that such analysis offers valuable insights into bridging architectural and machine learning research, fostering more effective and sustainable design strategies.

VII. ACKNOWLEDGMENT

We appreciate the reviewers for their helpful feedback. This work is supported by the National Key Research and Development Program (Grant No. 2024YFB4405600), the National Natural Science Foundation of China (Grant No. 62472086), the Basic Research Program of Jiangsu (Grant No. BK20243042), the Science and Technology Major Special Program of Jiangsu (No. BG2024010), and the Start-up Research Fund of Southeast University (Grant No. RF1028624005).

REFERENCES

- [1] Y. Chang, X. Wang, J. Wang, Y. Wu, L. Yang, K. Zhu, H. Chen, X. Yi, C. Wang, Y. Wang *et al.*, “A survey on evaluation of large language models,” *ACM Transactions on Intelligent Systems and Technology*, vol. 15, no. 3, pp. 1–45, 2024.
- [2] S. Han, J. Pool, J. Tran, and W. Dally, “Learning both weights and connections for efficient neural network,” *Advances in neural information processing systems*, vol. 28, 2015.
- [3] Z. Yao, S. Cao, W. Xiao, C. Zhang, and L. Nie, “Balanced sparsity for efficient dnn inference on gpu,” in *Proceedings of the AAAI conference on artificial intelligence*, vol. 33, no. 01, 2019, pp. 5676–5683.
- [4] S. Yang, Y. Sheng, J. E. Gonzalez, I. Stoica, and L. Zheng, “Post-training sparse attention with double sparsity,” *arXiv preprint arXiv:2408.07092*, 2024.
- [5] F. Farshchi, Q. Huang, and H. Yun, “Integrating nvidia deep learning accelerator (nvda) with risc-v soc on firesim,” in *2019 2nd Workshop on Energy Efficient Machine Learning and Cognitive Computing for Embedded Applications (EMC2)*. IEEE, 2019, pp. 21–25.
- [6] Y.-H. Chen, T. Krishna, J. S. Emer, and V. Sze, “Eyeriss: An energy-efficient reconfigurable accelerator for deep convolutional neural networks,” *IEEE journal of solid-state circuits*, vol. 52, no. 1, pp. 127–138, 2016.
- [7] J. Albericio, P. Judd, T. Hetherington, T. Aamodt, N. E. Jerger, and A. Moshovos, “Cnvlutin: Ineffectual-neuron-free deep neural network computing,” *ACM SIGARCH Computer Architecture News*, vol. 44, no. 3, pp. 1–13, 2016.
- [8] S. Liu, P. Li, J. Zhang, Y. Wang, H. Zhu, W. Jiang, S. Tang, C. Chen, Q. Liu, and M. Liu, “16.2 a 28nm 53.8 tops/w 8b sparse transformer accelerator with in-memory butterfly zero skipper for unstructured-pruned nn and cim-based local-attention-reusable engine,” in *2023 IEEE International Solid-State Circuits Conference (ISSCC)*. IEEE, 2023, pp. 250–252.
- [9] O. Mutlu, J. Stark, C. Wilkerson, and Y. N. Patt, “Runahead execution: An alternative to very large instruction windows for out-of-order processors,” in *The Ninth International Symposium on High-Performance Computer Architecture, 2003. HPCA-9 2003. Proceedings*. IEEE, 2003, pp. 129–140.
- [10] A. Naithani, S. Ainsworth, T. M. Jones, and L. Eeckhout, “Vector runahead,” in *2021 ACM/IEEE 48th Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 2021, pp. 195–208.
- [11] N. Jouppi, “Improving direct-mapped cache performance by the addition of a small fully-associative cache and prefetch buffers,” in *[1990] Proceedings. The 17th Annual International Symposium on Computer Architecture*, 1990, pp. 364–373.
- [12] S. Srinath, O. Mutlu, H. Kim, and Y. N. Patt, “Feedback directed prefetching: Improving the performance and bandwidth-efficiency of hardware prefetchers,” in *2007 IEEE 13th International Symposium on High Performance Computer Architecture*, 2007, pp. 63–74.
- [13] P. Michaud, “Best-offset hardware prefetching,” in *2016 IEEE International Symposium on High Performance Computer Architecture (HPCA)*, 2016, pp. 469–480.
- [14] Y. Ishii, M. Inaba, and K. Hiraki, “Access map pattern matching for data cache prefetch,” in *Proceedings of the 23rd International Conference on Supercomputing*, ser. ICS ’09. New York, NY, USA: Association for Computing Machinery, 2009, p. 499–500. [Online]. Available: <https://doi.org/10.1145/1542275.1542349>
- [15] S. Somogyi, T. Wenisch, A. Ailamaki, B. Falsafi, and A. Moshovos, “Spatial memory streaming,” in *33rd International Symposium on Computer Architecture (ISCA’06)*, 2006, pp. 252–263.
- [16] M. Shevgoor, S. Koladiya, R. Balasubramanian, C. Wilkerson, S. H. Pugsley, and Z. Chishti, “Efficiently prefetching complex address patterns,” in *2015 48th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2015, pp. 141–152.
- [17] M. Hashemi, O. Mutlu, and Y. N. Patt, “Continuous runahead: Transparent hardware acceleration for memory intensive workloads,” in *2016 49th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 2016, pp. 1–12.
- [18] A. Naithani, J. Feliu, A. Adileh, and L. Eeckhout, “Precise runahead execution,” in *2020 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 2020, pp. 397–410.
- [19] A. Naithani, J. Roelandts, S. Ainsworth, T. M. Jones, and L. Eeckhout, “Decoupled vector runahead,” in *Proceedings of the 56th Annual IEEE/ACM International Symposium on Microarchitecture*, 2023, pp. 17–31.
- [20] P. A. Tew, “An investigation of sparse tensor formats for tensor libraries,” M.Eng. Thesis, Massachusetts Institute of Technology, Cambridge, MA, Jun 2016. [Online]. Available: <http://tensor-compiler.org/files/tew-meng-thesis-sparse.pdf>
- [21] K. Kanellopoulos, N. Vijaykumar, C. Giannoula, R. Azizi, S. Koppula, N. M. Ghiasi, T. Shahroodi, J. G. Luna, and O. Mutlu, “Smash: Co-designing software compression and hardware-accelerated indexing for efficient sparse matrix operations,” in *Proceedings of the 52nd annual IEEE/ACM international symposium on microarchitecture*, 2019, pp. 600–614.
- [22] R. Hameed, W. Qadeer, M. Wachs, O. Azizi, A. Solomatnikov, B. C. Lee, S. Richardson, C. Kozyrakis, and M. Horowitz, “Understanding sources of inefficiency in general-purpose chips,” in *Proceedings of the 37th annual international symposium on Computer architecture*, 2010, pp. 37–47.
- [23] H. Genc, S. Kim, A. Amid, A. Haj-Ali, V. Iyer, P. Prakash, J. Zhao, D. Grubb, H. Liew, H. Mao *et al.*, “Gemmini: Enabling systematic deep-learning architecture evaluation via full-stack integration,” in *2021 58th ACM/IEEE Design Automation Conference (DAC)*. IEEE, 2021, pp. 769–774.
- [24] S. V. Crowder and M. D. Hamilton, “An ewma for monitoring a process standard deviation,” *Journal of Quality Technology*, vol. 24, no. 1, pp. 12–21, 1992.
- [25] A. Samajdar, Y. Zhu, P. Whatmough, M. Mattina, and T. Krishna, “Scale-sim: Systolic cnn accelerator simulator,” *arXiv preprint arXiv:1811.02883*, 2018.
- [26] H. Zhang, A. Ning, R. B. Prabhakar, and D. Wentzlaff, “Llmcompass: Enabling efficient hardware design for large language model inference,” in *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 2024, pp. 1080–1096.
- [27] I. Hur and C. Lin, “Memory prefetching using adaptive stream detection,” in *2006 39th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO’06)*. IEEE, 2006, pp. 397–408.
- [28] X. Yu, C. J. Hughes, N. Satish, and S. Devadas, “Imp: Indirect memory prefetcher,” in *Proceedings of the 48th International Symposium on Microarchitecture*, 2015, pp. 178–190.
- [29] Z. Niu, G. Zhong, and H. Yu, “A review on the attention mechanism of deep learning,” *Neurocomputing*, vol. 452, pp. 48–62, 2021.
- [30] A. Roy, M. Saffar, A. Vaswani, and D. Grangier, “Efficient content-based sparse attention with routing transformers,” *Transactions of the Association for Computational Linguistics*, vol. 9, pp. 53–68, 2021.
- [31] Z. Chen, Y. Deng, Y. Wu, Q. Gu, and Y. Li, “Towards understanding mixture of experts in deep learning,” *arXiv preprint arXiv:2208.02813*, 2022.
- [32] N. Du, Y. Huang, A. M. Dai, S. Tong, D. Lepikhin, Y. Xu, M. Krikun, Y. Zhou, A. W. Yu, O. Firat *et al.*, “Glam: Efficient scaling of language models with mixture-of-experts,” in *International Conference on Machine Learning*. PMLR, 2022, pp. 5547–5569.
- [33] M. Gadelha, R. Wang, and S. Maji, “Multiresolution tree networks for 3d point cloud processing,” in *Proceedings of the European Conference on Computer Vision (ECCV)*, 2018, pp. 103–118.
- [34] A. V. Phan, M. Le Nguyen, Y. L. H. Nguyen, and L. T. Bui, “Dgcnn: A convolutional neural network over large-scale labeled graphs,” *Neural Networks*, vol. 108, pp. 533–543, 2018.
- [35] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Lio, and Y. Bengio, “Graph attention networks,” *arXiv preprint arXiv:1710.10903*, 2017.
- [36] T. N. Kipf and M. Welling, “Semi-supervised classification with graph convolutional networks,” in *International Conference on Learning Representations (ICLR)*, 2017.
- [37] D. Zhang, J. Yan, K. Polat, A. Alhudhaif, and J. Li, “Multimodal joint prediction of traffic spatial-temporal data with graph sparse attention mechanism and bidirectional temporal convolutional network,” *Advanced Engineering Informatics*, vol. 62, p. 102533, 2024.
- [38] Z. Zhang, Y. Sheng, T. Zhou, T. Chen, L. Zheng, R. Cai, Z. Song, Y. Tian, C. Ré, C. Barrett *et al.*, “H2o: Heavy-hitter oracle for efficient generative inference of large language models,” *Advances in Neural Information Processing Systems*, vol. 36, pp. 34661–34710, 2023.
- [39] S. Brahmabhatt, V. Mehta, and S. Ghosh, “Minkowski sar-unet3d for point cloud semantic segmentation.”
- [40] J. Wang, W. Li, M. Zhang, and J. Chanussot, “Large kernel sparse convnet weighted by multi-frequency attention for remote sensing scene understanding,” *IEEE Transactions on Geoscience and Remote Sensing*, vol. 61, pp. 1–12, 2023.
- [41] W. Fedus, B. Zoph, and N. Shazeer, “Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity,” *Journal of Machine Learning Research*, vol. 23, no. 120, pp. 1–39, 2022.